

Script de soutenance

SeemsLegit - Plateforme de génération d'agents Mythic

M1 Cybersécurité · Oteria · Juin 2026

Harouna Coulibaly

Agent Red Team · Architecture · C99

Slides : 01, 02, 03, 08, 13

Killian Prin-Abeil

Malware Dev · Loaders · Profils C2

Slides : 09, 10, 11, 15

Jeremy Diaz

Malware Dev · Mythic · Plateforme

Slides : 04, 05, 06, 07, 12, 14

Durée totale estimée : 20 minutes · Questions jury : 10-15 minutes
16 slides · Ce document est un guide - adaptez le débit selon les réactions du jury.

Introduction

Slides 01-02 · 2 min

01 Titre - SeemsLegit

TOUS

~30 s

Harouna : Bonjour, nous sommes l'équipe SeemsLegit. Je suis Harouna Coulibaly.

Killian : Killian Prin-Abeil.

Jeremy : Et Jeremy Diaz. Nous sommes en M1 Cybersécurité à Oteria, et nous allons vous présenter notre projet de fin d'année : une plateforme de génération d'agents Mythic personnalisés, conçue pour contourner les solutions de détection actuelles.

02 Présentation de l'équipe

TOUS

~1 min 30

Harouna : Je suis Harouna. Dans ce projet j'ai pris en charge l'architecture globale de la plateforme, et le développement de l'agent Kratos en C99.

Killian : Moi c'est Killian. Je me suis concentré sur le développement malware en Nim - l'agent Aphrodite et le loader Hephaestus - et sur les profils C2 Chess.com et Notion.

Jeremy : Et moi Jeremy. J'ai développé les composants Go, intégré Mythic, et construit le Code Generator qui est la pièce centrale de la plateforme. J'ai aussi assuré les tests de bout en bout.

Note : Rester naturel, ne pas réciter. Le jury veut entrer dans le sujet rapidement.

Contexte

Slide 03 · 2 min

03 Contexte Red Team & Trilemme

HAROUNA

~2 min

Harouna : Le point de départ, c'est une réalité du Red Team aujourd'hui. Les frameworks C2 connus - Mythic, Havoc, Cobalt Strike - ont tous leurs signatures dans les bases EDR et antivirus. Un agent Mythic par défaut posé sur une machine cible est détecté en quelques secondes.

Face à ça, on a trois options, et aucune n'est satisfaisante seule.

La première : utiliser un agent Mythic par défaut. Rapide, mais détecté immédiatement.

La deuxième : développer un implant custom à la main pour chaque engagement. Efficace, mais ça prend des jours, ça exige une expertise rare, et ce n'est pas reproductible.

La troisième : passer par un packer ou crypter générique. Le problème : ils sont eux-mêmes signés dans les bases EDR et ne touchent pas au comportement runtime.

Aucune de ces trois options ne combine furtivité, efficacité et reproductibilité. C'est ce vide qu'on a voulu combler.

Transition : « Notre réponse à ce trilemme, c'est SeemsLegit. » -> Jeremy.

SeemsLegit

Slides 04-05 · 2 min

04 SeemsLegit - La plateforme

JEREMY

~1 min

Jeremy : SeemsLegit, c'est une plateforme web qui génère des agents Mythic où chaque build produit un binaire unique. Même code source, hashes différents à chaque compilation.

Concrètement : 4 agents développés from scratch, 2 loaders Windows, 3 profils C2 - dont deux qui utilisent des services légitimes comme canal covert. Et une plateforme unifiée qui orchestre tout avec un CLI.

L'idée centrale : le polymorphisme par build. On ne modifie pas le code à la main - la plateforme génère la variante automatiquement.

Transition : « Pour le pilotage du projet, Jeremy. »

05 Gestion de projet

JEREMY

~45 s

Jeremy : On a fonctionné avec un Kanban hébergé - visible sur la slide. Chaque tâche avait un responsable, une priorité et un état clair. Des points réguliers pour synchroniser les avancées sur la plateforme, les agents et les tests. Un lab partagé pour que tout le monde travaille dans le même environnement. Et une mesure continue sur VirusTotal, Defender et Elastic EDR dès le début.

Transition : « Voici l'architecture qui supporte tout ça. »

Architecture & Agents

Slides 06-07 · 3 min

06 Architecture de la plateforme

JEREMY

~1 min 30

Jeremy : L'architecture est construite autour de quatre services Docker Compose : un reverse proxy nginx, le frontend React, l'API FastAPI avec auth JWT, et PostgreSQL. Mythic C2 est intégré en service séparé. Les profils C2 sont eux aussi des services indépendants - on peut les échanger sans toucher à l'agent.

Tout se pilote via le CLI `seemslegit`. Les commandes clés : `seemslegit install` pour bootstrapper, `seemslegit up` pour démarrer, `seemslegit config init` pour initialiser les profils et Mythic. Un point d'entrée unique pour toute l'infrastructure.

Note : Pointer le schéma d'architecture si des questions portent sur les connexions entre services.

Transition : « Cette architecture supporte quatre agents que nous avons développés from scratch. »

07 Les 4 agents Mythic

JEREMY

~1 min

Jeremy : On a quatre agents, chacun dans un langage différent selon la cible et le besoin.

Kratos en C99, Windows - le plus furtif, avec indirect syscalls via HellHall. Morpheus en Go, Windows - compilation statique, chaîne de build obfusquée. Hermes en Python, Linux - rapide à déployer, pratique pour le scripting et la post-exploitation. Et Aphrodite en Nim, cross-platform - trois techniques d'injection et un loader BOF intégré.

Tous les quatre s'enregistrent dans Mythic et exécutent des commandes via nos profils C2. On va zoomer sur deux d'entre eux.

Transition : « Harouna commence avec Kratos. »

Focus agents & loaders

Slides 08-10 · 6 min

08 Kratos - Focus C99

HAROUNA

~2 min

Harouna : Kratos, c'est notre agent C99 pour Windows x64. Ce qui le distingue : il utilise des indirect syscalls via HellHall pour contourner les hooks userland des EDR.

Quand Kratos a besoin d'appeler `NtAllocateVirtualMemory`, il ne hardcode pas le numéro de syscall. Il scanne dynamiquement les stubs dans `ntdll.dll` pour extraire le SSN - le System Service Number - à la volée. HellHall lit les opcodes du stub de chaque fonction pour en déduire le SSN dynamiquement. Aucun numéro hardcodé dans le binaire.

Pour l'évasion : ntdll est unhookée par remapping depuis le disque, AMSI et ETW sont patchés en mémoire, le PPID est usurpé pour apparaître comme enfant d'un process légitime, et on détecte les artefacts de sandbox avant de s'exécuter.

En post-exploitation : gestion des credentials avec `steal_token` et `runas`, déplacement latéral avec `spawn`, et pivot réseau via `ligolo` et `socks`.

Note : Si question sur HellHall : « On résout le SSN dynamiquement depuis `ntdll.dll` en scannant les opcodes du stub. C'est la même idée que Hell's Gate mais en indirect syscall - on ne saute pas directement dans le kernel, on passe par une trampoline `ntdll` non hookée. Le call stack reste cohérent avec du code Microsoft. »

Transition : « Notre deuxième agent full-featured, Aphrodite en Nim. Killian. »

09 Aphrodite - Focus Nim

KILLIAN

~1 min 30

Killian : Aphrodite, c'est notre agent Nim. Il cross-compile Linux vers Windows via `mingw-w64` et livre soit un EXE soit un shellcode position-indépendant.

Ce qui le distingue, c'est ses trois techniques d'injection ordonnées par furtivité croissante. `createre motethread` est la plus simple mais visible par les EDR. `queueapcthread` est plus discrète. Et `ntmapview` - `NtMapViewOfSection` - est la plus furtive : elle évite `WriteProcessMemory` en mappant directement une section en mémoire, beaucoup moins détectée.

Pour l'évasion : obfuscation des chaînes au build, AMSI et ETW désactivés avant chaque opération sensible, `sleep jitter` entre les beacons.

En post-exploitation : `execute_assembly` qui charge le CLR en mémoire avec bypass AMSI intégré, `inline_execute` pour les BOF et COFF x64, et `steal_token` pour l'impersonation.

Note : Si question sur le chiffrement : « AES-256 standard via la lib `nimcrypto`. On n'a pas réimplémenté la crypto - on s'est concentrés sur les mécanismes d'injection et d'évasion. »

Transition : « Pour déposer ces agents sur une cible, on a deux loaders. » -> Killian continue.

10 Loaders - Hephaestus & Atreus

KILLIAN**HAROUNA**

~2 min

Killian : On a deux loaders Windows qui partagent les mêmes primitives d'évasion mais avec des techniques d'injection distinctes pour varier le profil comportemental.

Hephaestus est en Nim - même stack qu'Aphrodite. Il embarque trois techniques : Early Bird APC, qui injecte le shellcode avant que le process atteigne son point d'entrée. Thread Hijacking, qui suspend un thread existant et modifie son contexte. Et Process Hollowing, qui remplace l'image PE d'un process créé suspendu.

Harouna : Atreus est en C++ - même stack que Kratos. Il ajoute des techniques complémentaires : Fiber Injection via les fibres Win32 pour éviter `CreateThread`. Module

Stomping qui écrase une DLL légitime déjà chargée. Et Remote Injection classique via VirtualAllocEx.

Les deux partagent le même flux d'évasion : sandbox check, remapping ntdll, patch ETW/AMSI, spawn avec PPID spoofing, injection, puis heap wipe pour effacer les traces du shellcode en RAM.

Transition : « Ces agents communiquent via trois profils C2. Killian. »

Profils C2

Slide 11 · 2 min

11 Profils C2 - Chess.com & Notion

KILLIAN

~2 min

Killian : On a trois profils C2. Le plus spectaculaire : Chess.com. Le payload est encodé en Base5 - les pièces P, N, B, R, Q correspondent aux valeurs 0 à 4. Ces valeurs sont converties en positions FEN, publiées comme de vraies parties d'échecs sur Chess.com. L'agent lit ces parties via l'API publique. Regardez le board sur la slide - chaque coup est un octet de commande échangé. Le trafic est indiscernable d'un joueur normal.

Pour Notion : on utilise une base de données publique comme queue de tasking. Les entrées sont des commandes, les réponses sont des mises à jour. L'agent interroge l'API officielle - trafic HTTPS 100% légitime.

Et en complément : un profil HTTP standard avec jitter configurable, kill date et user-agent spoofing.

Note : Laisser l'animation Chess.com tourner quelques secondes avant de parler. C'est visuellement fort - le jury va regarder le board.

Transition : « On passe maintenant à la démonstration. Jeremy. »

Démonstration & Résultats

Slides 12-14 · 4 min

12 Démo

JEREMY

~2 min

Jeremy : On va vous montrer la chaîne complète en action. La vidéo montre : la configuration d'un agent depuis l'interface web, la sélection du profil C2 et des options d'évasion, le build - quelques secondes et le binaire est prêt - et enfin le check-in dans Mythic avec l'exécution d'une commande.

Note : Lancer la vidéo directement. Pas besoin de narrer chaque étape si c'est visible à l'écran. Si la vidéo lag, décrire verbalement ce qui se passe.

Transition : « Pour valider l'efficacité de ce polymorphisme, on a mesuré sur VirusTotal. Harouna. »

13 VirusTotal

HAROUNA

~1 min

Harouna : La validation concrète. Un agent Mythic par défaut, sans modification, est détecté par 47 moteurs sur 72. C'est notre baseline.

Avec un binaire généré par SeemsLegit, on descend à 8 détections sur 72. Les moteurs les plus connus - CrowdStrike, SentinelOne, ESET, TrendMicro - passent au vert. Il en reste quelques-uns, notamment sur la détection comportementale.

C'est une réduction de 83% des détections sur la comparaison statique.

Note : Si question sur ceux qui restent : « Les moteurs qui persistent font de la détection comportementale. Le statique s'améliore avec nos variantes, le dynamique est abordé dans le bilan. »

Transition : « Les résultats complets. Jeremy. »

14 Ce qu'on a livré

JEREMY

~1 min

Jeremy : En synthèse : 4 agents from scratch connectés à Mythic, 2 loaders Windows, 3 profils C2. Chaque build génère un binaire distinct - 47 détections en baseline, 8 avec nos binaires.

La chaîne complète a été testée de bout en bout : build, scan VirusTotal, loader, exécution, check-in C2. C'est un prototype fonctionnel - face à un EDR comportemental comme Elastic, la base technique est là mais il reste des itérations à faire.

Transition : « Killian pour le bilan. »

Bilan

Slide 15 · 2 min

15 Bilan

KILLIAN

~2 min

Killian : Trois points sur les difficultés. La détection statique, on l'a réduite de 83% - mais le gain était variable selon le loader et la structure du binaire final, pas systématique. La détection dynamique est le vrai challenge : même avec un binaire propre statiquement, les injections mémoire laissent des traces comportementales. Elastic EDR dans notre lab reste un challenge. Et la validation EDR comportementale est arrivée trop tard dans le projet - on n'a pas eu le temps d'itérer autant qu'on aurait voulu.

Ce qu'on en retient : statique et dynamique sont deux problèmes fondamentalement différents. Séparer agent, loader et profil C2 en composants indépendants simplifie les tests. Et mesurer tôt change les priorités.

Pour la suite : automatiser les campagnes EDR pour tester en continu, améliorer l'OPSEC des loaders sur le runtime, et étendre les profils C2 à d'autres frameworks que Mythic.

SeemsLegit, c'est un prototype de laboratoire - il ne prétend pas être un outil de production Red Team. Mais il nous a permis de relier concrètement build, injection, C2

et tests AV/EDR dans une même chaîne, et de comprendre en pratique où se situent les vraies difficultés.

Merci pour votre attention. On est disponibles pour vos questions.

Note : Se repositionner tous les trois face au jury. Harouna peut reprendre la parole en premier si une question générale est posée.

Annexe - Questions anticipées du jury

Q : Pourquoi Mythic plutôt que développer votre propre C2 ?

KILLIAN

Mythic est un framework open-source mature qui nous permet de nous concentrer sur ce qui nous intéresse : les agents, les loaders et les profils C2. Développer un C2 complet aurait consommé la totalité du projet. Mythic nous donne l'infrastructure de commande, on apporte la couche furtivité.

Q : HellHall vs Hell's Gate - quelle est la différence ?

HAROUNA

Hell's Gate résout le SSN dynamiquement mais l'utilise en direct syscall - on saute directement dans le kernel. HellHall fait la même résolution mais en indirect syscall : on passe par une instruction `syscall` trouvée dans un stub `ntdll` non hooké. Le call stack reste cohérent avec du code Microsoft, ce qui trompe les EDR qui analysent le call stack.

Q : Chess.com ne peut-il pas détecter et bannir ce comportement ?

KILLIAN

Techniquement oui - si Chess.com analysait le contenu sémantique des parties. En pratique, nos parties sont syntaxiquement valides et respectent les règles des échecs. Un humain regardant la partie ne verrait rien d'anormal. Chess.com pourrait détecter un volume anormal depuis une IP, mais dans un vrai contexte Red Team on utilise des comptes variés et le trafic est dilué.

Q : Elastic EDR détecte encore - comment vous comptez y remédier ?

HAROUNA

La détection dynamique est le vrai challenge. Nos binaires sont propres statiquement mais les injections mémoire laissent des traces comportementales. Les pistes : utiliser des techniques moins connues comme `NtMapViewOfSection` avec des patterns mémoire normaux, ou du process doppelganging. Et intégrer les tests EDR comportementaux plus tôt dans le cycle pour avoir le temps d'itérer.

Q : Quel est le niveau de maturité opérationnelle du projet ?

JEREMY

C'est un prototype de laboratoire. Il démontre que la chaîne fonctionne de bout en bout - build, scan, injection, check-in C2 - et que le polymorphisme par build réduit

significativement les détections statiques. On ne l'a pas testé en conditions réelles d'engagement, et la détection comportementale reste un axe à travailler. C'est un outil de recherche et d'apprentissage, pas un outil de production.

Q : Pourquoi ne pas avoir utilisé un seul langage pour tous les agents ?

JEREMY

La diversité était intentionnelle. C99 pour Kratos donne un contrôle maximal sur les syscalls et la mémoire. Nim pour Aphrodite permet le cross-compile facile et une bonne interop C. Python pour Hermes est pratique pour Linux et le scripting rapide. Go pour Morpheus offre la compilation statique et un bon équilibre entre perf et lisibilité. Chaque cible a ses contraintes.

Q : Pourquoi AES-256 dans Aphrodite - vous l'avez implémenté vous-mêmes ?

KILLIAN

Non, on utilise la bibliothèque nimcrypto, une lib Nim standard. On n'a pas réimplémenté la crypto - on s'est concentrés sur les mécanismes d'injection et d'évasion, qui sont le coeur du projet. AES-256 pour le chiffrement des communications, primitives éprouvées.

Q : La plateforme web est-elle sécurisée - comment protégez-vous l'accès ?

JEREMY

Auth JWT avec tokens signés côté API FastAPI. Le frontend React ne stocke pas le token en localStorage - il passe par un cookie httpOnly. Nginx en reverse proxy gère le TLS. Dans le cadre du projet, l'infra est déployée en lab isolé - on n'a pas exposé la plateforme sur Internet. Pour un usage réel, on ajouterait un VPN ou un bastion devant.